MATTERS — TIA AI ASSISTANT

Phase 1: Read-Only Architecture Audit & Diagnostic Report

Scope: Voice, Personality, Reasoning, Speech Synthesis, & Safety Pipelines | Version 1.0

AUDIT EXECUTION METADATA

- Status: Completed under Strict Read-Only Mode (Zero files modified/deleted, zero code regressions).
- Target Application: Matters Learning App (React SPA + Express + Gemini AI Integration).
- Primary Objective: Safe, minimal-footprint architectural audit for Voice realism, Personality, and Answer depth.
- Target Roadmap: Browser Speech vs Gemini 2.5 Flash TTS vs ElevenLabs with Context-Aware Guardrails.

1. Existing Tia Architecture Inspection

Component inventory, handlers, service pipelines, and routes

Tia is integrated as a hybrid client-server interactive learning assistant. The frontend provides a floating interactive bubble with rich voice and text multimodal capabilities, while the server delegates requests to Google Gemini with semantic search and course curriculum retrieval.

Exact File Paths & Structural Inventory

- **Floating Launcher:** src/components/tia/TiaFloatingButton.tsx — Renders the draggable floating button, manages open/closed modal state, and animates speaking/listening wave pulses.
- **Assistant Modal & UI:** src/components/tia/TiaAssistantModal.tsx — 925-line complete modal orchestrating chat history, microphone controls, language selector (English/Hindi), audio wave animations, and follow-up countdown timer.
- **Voice & Speech Engine Hook:** src/lib/useTiaVoice.ts — 776-line custom hook managing Web Speech API recognition (webkitSpeechRecognition), speech synthesis playback, sentence chunking, mobile workarounds, and 4-second follow-up listening.
- **Client Semantic & State Hub:** src/services/tiaService.ts — Central dispatch managing local state, quick prompt actions, quiz answer evaluations, and routing text to either Express backend or client-side semantic engine.
- **Language Configuration:** src/services/tiaLanguageConfig.ts — Defines 4 supported language modes (English en-IN/en-US, Hindi hi-IN, Hinglish, Mixed) and preferred browser voice names.
- **Express API Gateway:** api/tia/chat.ts & server.ts (lines 35-95) — POST /api/tia/chat endpoint validating payloads, handling rate limiting, and invoking server-side Gemini.
- **Server-side AI Handler:** src/services/tiaAiHandler.ts — Direct Google Gen AI integration via @google/genai SDK, system prompt formulation, lesson context retrieval, and error fallbacks.
- **Curriculum & Search Registry:** src/services/tiaCourseRegistry.ts & src/services/tiaSemanticEngine.ts — Structured registry of all 42 micro-courses across 7 categories and client-side offline TF-IDF query match fallback.

End-to-End Execution Flow: Step-by-Step Trace

FLOW COMPARISON: TYPED VS VOICE INPUT

1. User Input Trigger: User types text OR presses the mic button. Mic uses Web Speech API to transcribe speech in real-time.
2. Unified Dispatch: Transcribed speech populates user message state and invokes the identical handleSendMessage() function.
3. Backend Delivery: Dispatched to POST /api/tia/chat with message, conversation history (last 6 turns), language, and user learning context.
4. Gemini Model Execution: Server executes gemini-2.5-flash with system persona prompt, returning structured AI response text.
5. UI Render & Speech Trigger: TiaAssistantModal renders Markdown output. If voice is enabled, Tia calls speakResponse() in useTiaVoice.ts.
6. Follow-up Window: Once speech playback finishes, onEnd callback starts a 4-second countdown window allowing conversational hands-free reply.

2. Microphone Behavior & Safety Audit

Wake-word, always-on inspection, and hands-free follow-up analysis

A rigorous code inspection of TiaAssistantModal.tsx and useTiaVoice.ts confirmed that Tia strictly obeys user privacy constraints:

- **Manual Mic Activation:** The microphone is only turned on when the user explicitly clicks the mic button or when the immediate post-speech follow-up window (4s) triggers startListening().
- **Wake-Word Absence:** There is NO wake-word detection, NO "Hey Tia" background listening daemon, and NO persistent audio capture when the modal is closed or conversation is idle.
- **Follow-Up Window:** Tia implements a 4000ms (4-second) follow-up listening timer (line 218 in useTiaVoice.ts) triggered exclusively upon speech synthesis end.
- **Auto-Termination:** The recognition automatically stops on silence or speech end, invoking recognition.stop() and clearing timers.

3. Voice Quality & Playback Defect Audit

Diagnosis of current audio engine, synthesis cutoff, and language switching

Tia currently relies entirely on the client browser SpeechSynthesis API (window.speechSynthesis). This creates significant architectural limitations across devices:

- **Synthesis Stopping After Few Words:** A known Chrome/Chromium bug terminates window.speechSynthesis utterances longer than 15 seconds. While useTiaVoice.ts attempts chunking (chunkTextIntoSentences), boundary splitting errors cause stuttering and dropouts.
- **Device & OS Voice Inconsistencies:** Browser speech uses whatever voice pack is locally installed. On Windows, Android, Linux, and iOS, the voice sounds robotic, metallic, or defaults to low-quality monotone voices.
- **Hindi / Hinglish Breakdown:** In Hindi mode, if the system lacks a native hi-IN engine, it attempts to read Devanagari text using an English phoneme engine, resulting in unintelligible garbled speech.
- **React Rerender Interference:** In TiaAssistantModal.tsx, when streaming text or updating UI state, excessive re-renders can trigger effect cleanups that call window.speechSynthesis.cancel(), cutting Tia off mid-sentence.
- **Overlapping Speech Requests:** Rapid clicks or simultaneous triggers can cause speech synthesis queues to collide or jam.

Evaluation of Future Voice Options (Option A vs Option B)

OPTION A: GEMINI 2.5 FLASH TTS (AUDIO OUTPUT MODALITY)

• Naturalness & Expressiveness: Native conversational pacing, emotional inflection, whisper/wit nuance, human-like cadence.
• Multilingual Native Support: Flawless Indian English (en-IN), clear conversational Hindi, and natural Hinglish code-switching.
• Architectural Alignment: Seamlessly integrates with the existing @google/genai SDK in server.ts without external 3rd-party vendors.
• Single-Roundtrip Advantage: Gemini can generate both reasoning text and synthesized PCM/WAV audio in a single API call.
• Implementation Complexity: Low-to-Medium (convert backend endpoint to stream base64 audio chunks to client Web Audio API).

OPTION B: ELEVENLABS REST/STREAMING TTS

• Naturalness & Expressiveness: Industry benchmark for studio-grade human voices with ultra-customizable voice IDs.
• Multilingual Support: ElevenLabs Multilingual v2 handles English and Hindi, though Hinglish slang occasionally requires phonetic tuning.
• Architectural Alignment: Requires separate vendor API key, additional network hop (Gemini -> Server -> ElevenLabs -> Client).
• Cost & Latency: Higher latency (two sequential LLM/TTS calls) and higher per-character pricing tiers.
• Implementation Complexity: Medium (additional SDK and credit quota management).

4. Prompt System, Knowledge Base & Personality Audit

Analysis of persona, reasoning depth, and course curriculum grounding

Tia's personality prompt is located in src/services/tiaAiHandler.ts (SYSTEM_PROMPT, lines 20-140). The prompt is well-structured but suffers from rigid hardcoded boundaries:

- **Current Persona Definition:** Tia is defined as "warm, encouraging, intellectually sharp, playful, witty, and concise". However, tone often collapses into dry informational summaries because witty few-shot demonstrations are absent.
- **Quiz & Humor Fallbacks:** When offline or when API errors occur, tiaService.ts falls back to static hardcoded jokes (lines 200-240) and rigid quizzes, breaking user immersion.
- **Context Clipping:** Context injection in tiaAiHandler.ts limits lesson snippets to 800 characters (lines 260-290), truncating complex subject nuances and leading to generic high-level answers.
- **Safety Guardrails Gap:** While financial and medical disclaimers are present, explicit edge-case jailbreak prevention and hallucination boundaries are not formally structured.

5. Safe Minimal-Change Upgrade Roadmap

Phased implementation blueprint preserving all working features

To deliver realistic voice, razor-sharp wit, and expert reasoning with zero regression risks, the upgrade should be executed in 4 sequential, reversible phases:

PHASE 1: PROMPT & REASONING ENHANCEMENT (ZERO RISK)

• Target Files: src/services/tiaAiHandler.ts
• Changes: Expand system prompt with 6 witty few-shot conversation examples, context-aware banter, and structured thinking steps.
• Safety: Formalize guardrail boundaries (stay in learning scope, decline dangerous queries wittily). Increase context window from 800 to 2,500 chars.
• Risk Level: Very Low (No frontend changes, no API route changes, instantly reversible).

PHASE 2: AUDIO SYNTHESIS UPGRADE (HIGH IMPACT)

• Target Files: server.ts, src/services/tiaAiHandler.ts, src/lib/useTiaVoice.ts
• Changes: Implement Gemini TTS audio streaming modality (or ElevenLabs proxy) on /api/tia/chat returning audio buffer.
• Safety Fallback: Keep window.speechSynthesis as graceful offline fallback if audio stream fails.
• Risk Level: Low (Graceful degradation guarantees speech is never broken).

PHASE 3: SMART FOLLOW-UP & VOICE CONVERSATION LOOP

• Target Files: src/lib/useTiaVoice.ts, src/components/tia/TiaAssistantModal.tsx
• Changes: Robust Web Audio API player with seamless transition into the 4-second listening window. Eliminate speech cancel collisions.
• Safety: Strict event listener cleanups on component unmount.
• Risk Level: Low.

PHASE 4: REGRESSION AUDIT & VERIFICATION MATRIX

• Verification: Test typed messages, manual mic button, follow-up countdown, course context grounding, and language switching.
• Backward Compatibility: Verify mobile Safari and Android Chrome compatibility.
• Risk Level: Zero.

AUDIT CONCLUSION & READINESS

The audit demonstrates that Matters has an exceptionally clean, well-modularized architecture. Tia can be upgraded to world-class voice realism and delightful conversational intelligence through targeted enhancements to the backend prompt and audio synthesis streaming pipeline, without disrupting any existing UI, database, or learning features.